

Algoritmi e Strutture Dati – 14/6/2024 – Parte A

Esercizio -1 Iscriverti allo scritto entro la scadenza.
In caso di inadempienza, -1 al voto finale.

Esercizio 0 Scrivere correttamente nome, cognome, numero di matricola, riga e colonna su tutti i fogli consegnati. Consegnare foglio A4 e foglio protocollo di bella. In caso di inadempienza, -1 al voto finale.

A1 – Complessità – Punti ≥ 8

Trovare un limite superiore e inferiore al costo computazionale del seguente algoritmo.

```

int super(int n)
    if n < 4 then
        return n
    else
        int b = 1
        for i = 1 to  $\lfloor n/5 \rfloor$  do
             $b = (b \cdot 2) \bmod 997$ 
        return b + super( $\lfloor n/2 \rfloor$ ) + super( $\lfloor n/4 \rfloor$ )

```

A2 – Big Fire – Punti ≥ 10

Si consideri una matrice intera A di dimensione $n \times n$, che rappresenta un'area geografica in cui le celle di valore 1 contengono bosco e le celle con valore 0 sono senza bosco. Se si sviluppa un incendio, si diffonde in questo modo:

- quando una cella contenente bosco prende fuoco, tutte le celle adiacenti prendono fuoco, sia che contengano bosco sia che siano senza;
- quando una cella senza bosco prende fuoco, tutte le celle adiacenti con bosco prendono fuoco, mentre le celle adiacenti senza bosco non prendono fuoco.

Scrivere un algoritmo

int bigfire(**int**[][] A , **int** n , **int** i , **int** j)

che prenda in input la matrice A , la sua dimensione n e le coordinate (i, j) di una cella e restituisca il numero di celle che prendono fuoco quando viene appiccato un incendio nella posizione (i, j) .

Discutere informalmente la correttezza dell'algoritmo e la sua complessità computazionale.

Per esempio, nella matrice di sinistra, un incendio che inizia in $(1, 1)$ si diffonde nelle caselle in grigio e la funzione dovrà restituire 8; nella matrice di destra, un incendio che inizia in $(1, 1)$ si diffonde nelle caselle in grigio e la funzione dovrà restituire 13.

1	1	1	1
0	0	0	0
0	0	0	0
1	1	1	1

1	1	1	1
0	0	0	1
0	0	0	0
0	0	1	1

A3 – Smaller Count – Punti ≥ 12

Scrivere un algoritmo

int[] smaller(**int**[] A , **int** n)

che prenda in input un vettore A contenente n interi distinti e restituisca un vettore *count* dove *count*[i] contiene il numero di elementi di $A[i + 1 \dots n]$ che sono più piccoli di $A[i]$:

$$\text{count}[i] = |\{A[j] \mid i < j \leq n \text{ and } A[j] < A[i]\}|$$

Discutere informalmente la correttezza dell'algoritmo e la sua complessità computazionale. Una soluzione $\Omega(n^2)$ verrà valutata 50%.

Per esempio, per il vettore $A = [1, 7, 2, 3, -1]$, la risposta è *count* = $[1, 3, 1, 1, 0]$.

Algoritmi e Strutture Dati – 14/6/2024 – Parte B

Esercizio -1 Iscriverti allo scritto entro la scadenza. In caso di inadempienza, -1 al voto finale.

Esercizio 0 Scrivere correttamente nome, cognome, numero di matricola, riga e colonna su tutti i fogli consegnati. Consegnare foglio A4 e foglio protocollo di bella. In caso di inadempienza, -1 al voto finale.

B1 – Corsa a ostacoli – Punti ≥ 8

Scrivere un algoritmo

```
int paths(boolean[][] A, int n)
```

che prenda in input una matrice di dimensione $n \times n$ e restituisca il numero di percorsi che permettono di andare dalla posizione (n, n) alla posizione $(1, 1)$, potendo andare solo in alto (riga-1) o a sinistra (colonna-1) e considerato che non è possibile attraversare celle della matrice il cui valore sia **false**.

Discutere informalmente la correttezza dell'algoritmo e la sua complessità computazionale. Potete assumere che $A[1][1] = A[n][n] = \text{true}$, ma non è strettamente necessario.

Per esempio, nella matrice seguente esistono due cammini che vanno da $(3, 3)$ a $(1, 1)$, quindi la funzione deve restituire 2.

T	T	T
T	F	T
T	T	T

B2 – Multi core – Punti ≥ 10

Supponiamo di avere k processori e n task da eseguire. Il tempo di esecuzione $len[i]$ del task i -esimo è memorizzato nel vettore len di dimensione n . L'obiettivo è assegnare i task ai k processori, in modo da minimizzare il tempo di esecuzione totale. Il tempo di esecuzione di un singolo processore è dato dalla somma dei tempi di esecuzione dei task assegnati ad esso, mentre il tempo di esecuzione totale è pari al massimo dei tempi di esecuzione di tutti i processori (i.e., l'istante in cui l'ultimo processore termina di eseguire).

Si considerino i tre seguenti algoritmi greedy:

- I task vengono esaminati nell'ordine in cui sono memorizzati nel vettore, e ciascun task viene assegnato al processore con il tempo di esecuzione corrente minore.
- I task vengono esaminati in ordine crescente di tempo di esecuzione, e ciascun task viene assegnato al processore con il tempo di esecuzione corrente minore.
- I task vengono esaminati in ordine decrescente di tempo di esecuzione, e ciascun task viene assegnato al processore con il tempo di esecuzione corrente minore.

Per tempo di esecuzione corrente, si intende la somma dei tempi di esecuzione che vengono mano a mano assegnati a un processore.

Per ogni algoritmo, dimostrare la correttezza o fornire un controesempio.

B3 – Magic square – Punti ≥ 12

Un quadrato magico è una matrice $n \times n$ contenente numeri da 1 a n^2 tale che i numeri siano tutti distinti e le somme dei valori su tutte le righe, tutte le colonne e in entrambe le diagonal di diano tutte lo stesso valore.

Per esempio, quello che segue è l'unico quadrato magico di dimensione 3, senza considerare rotazioni e simmetrie.

2	7	6
9	5	1
4	3	8

Assumendo di avere a disposizione una funzione

```
boolean isMagic(int[][] A, int n)
```

che restituisce **true** se e solo se la matrice A contiene un quadrato magico $n \times n$ (con costo $\Theta(n^2)$), scrivere un algoritmo

```
printMagic(int n)
```

che stampa tutti i quadrati magici di dimensione $n \times n$.

Discutere informalmente la correttezza dell'algoritmo e la sua complessità computazionale.